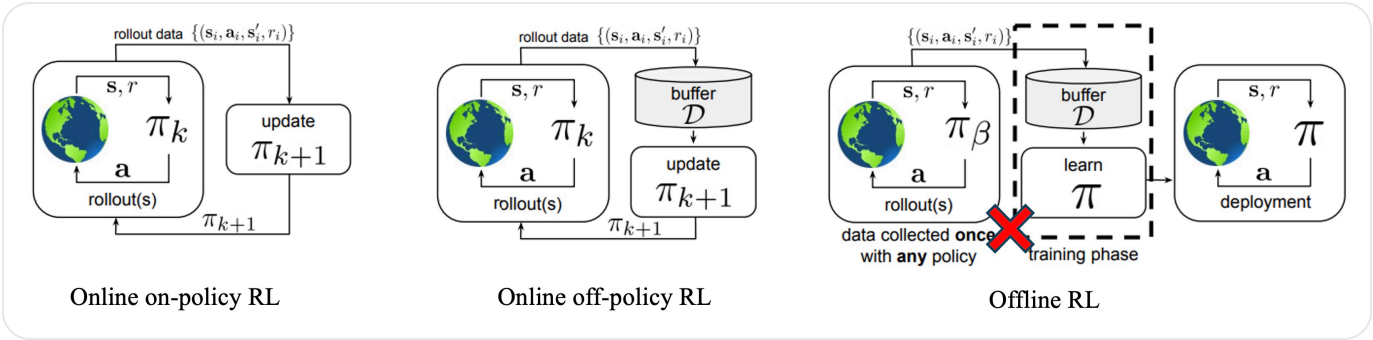
A Practical Way to Classify RL: by Data Source and by Update Schedule

Reinforcement learning can look like a forest of algorithms, but two simple axes make the landscape manageable. The first is the **data source**—where trajectories come from. The second is the **update schedule**—how often we alternate **policy evaluation** (estimating the value of a policy) and **policy improvement** (updating the policy using that estimate). This perspective provides a consistent vocabulary for both classic and modern methods, and it maps cleanly onto real constraints in robotics such as safety, sample efficiency, and reliability.

Data-source perspective



On-policy vs Off-policy vs Offline RL.

On-policy learning collects trajectories using the current policy and updates on those fresh rollouts, which keeps the training distribution aligned but can be sample-hungry. **Off-policy** learning reuses past experience from a replay buffer gathered by some behavior policy, trading alignment for efficiency and requiring corrections or regularization. **Offline RL** goes further by training from a fixed dataset without any

additional interaction. With this setup in mind, we can write unified forms for *policy evaluation* and *policy improvement* that cover a wide range of algorithms.

Unified policy evaluation & improvement

Policy evaluation (unified)

$$\hat{Q}^{k+1} = \arg\min_{Q} \mathbb{E}_{(s,a)\sim w}\Big[\Big(\underbrace{r + \gamma\, \mathbb{E}_{s'|s,a}\, \mathbb{E}_{a'\sim\pi^k(\cdot|s')}[\hat{Q}(s',a')]}_{T^{\pi^k}\hat{Q}} - Q(s,a)\Big)^2\Big].$$

Policy improvement (unified)

$$\pi^{k+1} = \arg\max_{\pi} \mathbb{E}_{s\sim\rho,\, a\sim\pi}\big[\hat{Q}^{k+1}(s,a)\big] - \beta\, \mathbb{E}_{s\sim\rho}\big[D_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})\big].$$

on-policy

- **Data/source:** trajectories from the current policy π^k (distribution d^{π^k}).
- **Evaluation:** choose $w = d^{\pi^k}$; realize $\mathbb{E}_{a'\sim\pi^k}$ by direct sampling $a' \sim \pi^k(\cdot|s')$ (SARSA) or expected backup $\sum_{a'} \pi^k(a'|s')\, \hat{Q}(s',a')$ (Expected SARSA).
- **Improvement:** choose $\rho = d^{\pi^k}$, $\pi_{\mathrm{ref}} = \pi^k$ → TRPO/PPO-style trust region.

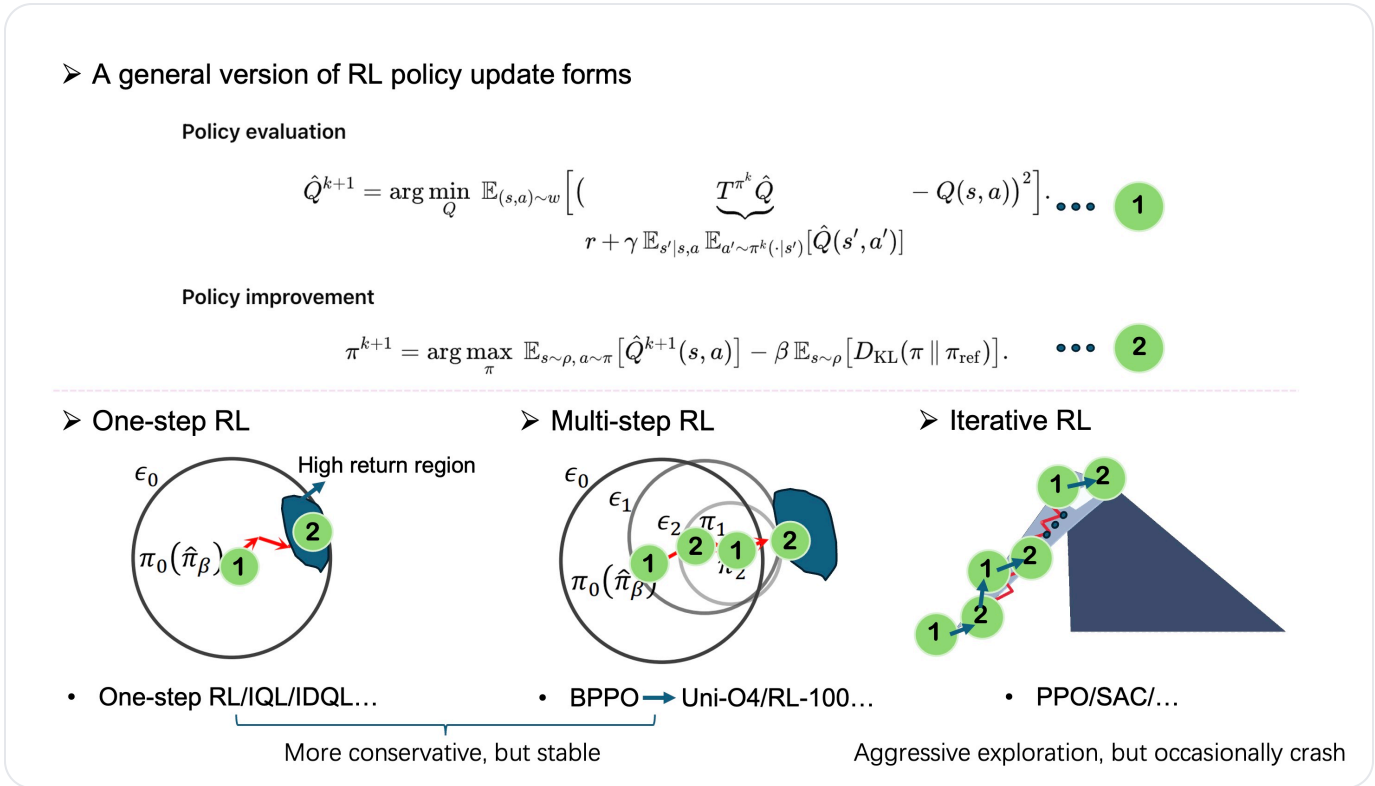
off-policy / offline

- **Data/source:** replay/offline from behavior μ (distribution d^{μ}), with $\mu \neq \pi^k$.

- **Evaluation:** choose $w = d^{\mu}$; approximate $\mathbb{E}_{a' \sim \pi^k}$ via *IS* ($\delta_t = r_t + \gamma \rho_{t+1} \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t)$, $\rho_{t+1} = \frac{\pi^k(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})}$), *traces/projected* (TD(λ), Retrace, V-trace, IQL/FQE), or *control backup* $y = r + \gamma \max_{a'} \hat{Q}(s', a')$ (Q-learning).
- **Improvement:** choose $\rho = d^{\mu}$, $\pi_{\text{ref}} = \mu$ → behavior-regularized updates (BRAC/AWAC).
  SAC mapping: use soft Bellman $\mathbb{E}_{a' \sim \pi}[\min_i Q(s', a') - \alpha \log \pi(a'|s')]$, and set $\pi_{\text{ref}} = \text{Uniform}$, $\beta = \alpha$ (entropy regularization).

↑ Back to top

Update-schedule perspective



Iterative vs multi-step vs one-step schedules.

Focusing solely on how often we alternate evaluation and improvement yields a second, surprisingly useful taxonomy. If we repeat these components many times with small steps and ongoing interaction, we get **iterative RL**, which is the typical online setting. If we perform one evaluation followed by a single improvement on a fixed dataset or with data flywheel, we get a **one-step RL** method—a natural choice when interaction is severely limited—though it can be overly conservative and suffer from *exploration isolation* at deployment. Between the two sits **multi-step RL**: a handful of evaluation–improvement rounds with safeguards such as trust regions, behavior regularization, or offline policy evaluation (OPE) gates. This middle ground is often the most pragmatic balance—less timid than one-step, yet less risky than fully aggressive iteration—providing a better trade-off between conservatism (stability) and exploration (which can occasionally crash).

Robotic foundation models and the data flywheel

This schedule-centric view lines up well with the training practice of modern robotic foundation models such as Generalist's newly announced GEN-0 and Pi's pi_0.5. These systems grow through a **data flywheel**: continually aggregating new tasks and scenarios into a unified corpus, then retraining or fine-tuning at scale. In that setting, **multi-step** updates are a natural fit. Each cycle makes a small, safety-gated improvement—conservative enough to avoid distribution crashes, yet with room for controlled exploration as the corpus expands.

As the model strengthens and approaches bottlenecks—either because it must **surpass human ceilings** on certain capabilities or **align tightly** with human performance—it's appropriate to hand off to **iterative** online RL on targeted objectives. There, frequent evaluate–improve alternation and carefully designed interactions provide the sharper feedback needed to push the frontier. We have seen this play out in practice in *rl-100*–style setups: multi-step updates deliver reliable gains on limited data, and a small

amount of iterative online RL on selected metrics can raise the ceiling further without compromising safety.

References

RL-100: Performant Robotic Manipulation with Real-World Reinforcement Learning
Kun Lei*, Huanyu Li*, Dongjie Yu*, Zhenyu Wei*, Lingxiao Guo, Zhennan Jiang, Ziyu Wang, Shiyu Liang, Huazhe Xu.
project page / arXiv / code / twitter / blog / Zhihu

Uni-O4: Unifying Online and Offline Deep Reinforcement Learning with Multi-Step On-Policy Optimization
Kun Lei, Zhengmao He*, Chenhao Lu*, Kaizhe Hu, Yang Gao, Huazhe Xu.
International Conference on Learning Representations (ICLR), 2024
project page / arXiv / code / twitter

Behavior Proximal Policy Optimization
Zifeng Zhuang*, Kun Lei*, Jinxin Liu, Donglin Wang, Yilang Guo.
International Conference on Learning Representations (ICLR), 2023
paper / arXiv / code

↑ Back to top